

ADSA Assignment 2 Report

In this report, I will explain the technical details of the code, as well as how I wrote it and how I tested it.

My implementation of an AVL tree, with insertion, deletion, and traversal, was all done in C++ with no assistance from LLMs.

Trees are structures made up of nodes, so the most foundational element of my code was the node class. It is under the `=== AVL NODE ===` heading. I had private variables and public functions in accordance with OOP principles.

Each node has a key, the value that we are putting in the tree, a height, which represents how far from up the tree the node is (starting at 1 and increasing with each generation below it), and two child nodes titled Left and Right. These are pointers for the sake of efficiency.

I have a function to initialise a node with a given key value, which sets the key to the new key value, height to 1, and children to nullptrs. I then had get functions for each of the four private variables, as well as one for the balance.

The balance is an important value that comes up when balancing a tree and needs to be recalculated whenever it is required. Mathematically, it is the height of the left child minus the height of the right child, but it looks more complicated in code as it requires handling the cases where the node in question hasn't got any children or only has one. If these cases are not handled, a segmentation fault occurs, as the program is trying to access the height of a nullptr.

Following on, I have four simple set functions that allow the values of keys, heights, and children to be altered.

The next part of my code is the input function, under the `=== INPUT ===` heading. This is a simple function that takes in the standard input and breaks it up into each "word" (in this case combinations of a single letter and a number, or the finishing move). It stops reading once the finishing move is detected. There is no input sanitization as the task sheet specifically says we do not need to check for this.

Next is the rotation helper function set. I will come back to this, in favour of the insert function. This is under the banner `=== INSERT ===`. The insert function requires a node pointer and a key integer as input and returns a node pointer. This node pointer is the root node of the sub-tree, and eventually the entire tree, and it is imperative that any changes to the root node are caught by the main function, as the root node may change each

insertion. If these changes are not caught, then the insertion would only happen in the subtree that that node heads. This is likely to result in an unbalanced tree and could cause duplicate values.

The insert function is recursive. This method is the simplest as recursion essentially “tracks” where the function has searched to place the node, which is important as on the way back up the tree, heights and balances need to be recalculated and checked. I have my base cases where the current node is a nullptr, which means that the new node can be placed there. If the current node is not a nullptr, the function is called again for one of either the left or right subtrees, depending on the comparative key values of the nodes. Importantly, the left or right child of the current node is set to the return value, as the return value is the root node of the subtree, which can change upon insertion. There is also a case here for when the current node has an equal key value to the new key, in which case we simply return current, as we do not insert duplicate values.

After these steps, there are a series of lines that are called as the function is travelling back up to the root of the tree. At each node of the initial traversal, the height of the current node is recalculated and set, with care for nullptrs. Then, the balance of the current node is checked. From here, there are five cases:

1. Left-left case: if the balance is greater than 1, meaning it is heavy on the left side, and the key is smaller than the left child's key, then the subtree is rotated to the right using the rotateRight function.
2. Right-right case: if the balance is less than -1, meaning it is heavy on the right side, and the key is greater than the right child's key, then the subtree is rotated left using the rotateLeft function.
3. Left-right case: if the balance is greater than one, meaning it is heavy on the left side, and the key is greater than the left child's key, then the left child node is rotated left, and the current node is rotated right.
4. Right-left case: if the balance is less than -1, meaning it is heavy on the right side, and the key is less than the right child node's key, then the right child node is rotated right and the current node is rotated left.
5. The balance is within the healthy range of $[-1, 1]$, and no rotation is required.

These are conducted with the functions under the `=== ROTATIONS ===` heading. The insert function simply finishes up by returning the current node, which will, in the main function, be the root node.

Under the `=== ROTATIONS ===` heading, there is a helper function to find the height of a node with care for the `nullptr` case. Without this helper function, simply using the node class's height finder would have made the code more verbose.

The right rotation takes the unbalanced node, and interacts with two generations below it. Let A be an unbalanced node with left child node B, which in turn has a left child node X and right child node C. First, B will become the root with left child X and right child A. Then, its former right child C will become A's left child. The process with left rotation is almost identical but in the opposite direction. It is with combinations of these two functions that all AVL trees can become balanced.

Following all of this is the `deleteNode` function, under the `=== DELETE ===` heading. For the same reasons as insertion, this function returns the root node pointer. It takes the root node pointer and a key value to be deleted.

This function behaves very similarly to insertion: the tree is traversed recursively until the node in question is found (or not found, at which point it returns with no change), and then deleted, and the tree rebalanced. There are, of course, special cases, as deleting a node requires slightly more effort than inserting one. If the node has no children, it is simply deleted. If the node has one child, then the child replaces the now-deleted node, returning it as it is now the root of the sub-tree. Where there are two children, the left-most child of the right subtree is found and placed into the empty gap where the deleted node resided. This left-most child will always have between 0-1 children, so deleting it and moving it up is not a complicated problem, and can be done using the function I'm describing presently. Then, the key for the node being deleted is simply changed to the newfound node's key. Additionally, if the tree only had the one node and the node is now deleted, it returns a `nullptr`.

The height and balance updating is almost identical to that of the insert function, and so I won't describe it again.

Following deletion is the traversal set of functions. Under the heading `=== TRAVERSALS ===` are the functions `preOrder`, `inOrder` and `postOrder`. These functions each extract every key from the tree and place them into a vector of integers which is then returned. Each find these values recursively. The difference between them all is the order in which they find the values. The `preOrder` function finds the root node, then the left sub-tree's nodes, and then the right sub-tree's nodes. As this is recursive, when the function is called on the left or right sub-tree, the root node of the sub-tree is called first, and then so on and so forth until childless nodes are found. The `postOrder` function is similar, except that the root node is found last, so the left subtree is investigated first, then the right, then the root node. The

inOrder function finds the left-subtree first, then the root node, and then the right sub-tree, which due to the ordering of AVL trees means it outputs the key values in numerical order.

Finally, the main function, under the `=== MAIN ===` heading, creates a root node `nullptr`. Then, the input is collected using a vector of strings and the input function. This input is then checked through according to the first letter of each element, and the required insert or `deleteNode` function is called. A vector of integers is created, and the tree traversed according to the value of the final element of the input vector. This vector is then printed according to the specifications, with “EMPTY” being printed if the tree is empty.

For testing, I struggled initially with getting the recursion for insertion correct, so I used the following input often: “A2 A4 A3 A35 A4 A1 PRE”. Initially, I used this to check that I was getting each value inputted correctly and not getting duplicates. Eventually, it became very helpful in handling the rotation, once I got to that point. My struggle was that I wasn’t picking up the values of the root node correctly, so I kept getting duplicates and occasionally garbage values. Once I got this input working, I found every other input working as expected. I would check by using the AVL tree visualiser:

<https://www.cs.usfca.edu/~galles/visualization/AVLtree.html>. This allowed me to double check what I believed the rotations should look like and thus be certain that my expected output was correct. I also had some trouble after implementing the `deleteNode` function, as I hadn’t been careful about `nullptrs` when checking height. As a result, when I attempted to delete a node that wasn’t present in the tree, I would get a segmentation fault.

The last few test cases were specifically designed to prompt each of the four rotations, via both insertion and deletion. As such, I only tested them with PRE output.

My test cases are detailed below:

Input	Expected Output	Output
A2 A4 A3 A35 A4 A1 PRE	3 2 1 4 35	3 2 1 4 35
A2 A4 A3 A35 A4 A1 IN	1 2 3 4 35	1 2 3 4 35
A2 A4 A3 A35 A4 A1 POST	1 2 35 4 3	1 2 35 4 3
A5 D5 PRE	EMPTY	EMPTY
A5 D4 PRE	5	5
A5 A7 A3 D1 D5 A35 D9 A2 A4 D2 PRE	7 3 4 35	7 3 4 35
A5 A7 A3 D1 D5 A35 D9 A2 A4 D2 IN	3 4 7 35	3 4 7 35
A5 A7 A3 D1 D5 A35 D9 A2 A4 D2 POST	4 3 35 7	4 3 35 7
A1 A2 A3 PRE (right right)	2 1 3	2 1 3
A3 A2 A1 PRE (left left)	2 1 3	2 1 3

A1 A3 A2 PRE (right left)	2 1 3	2 1 3
A3 A1 A2 PRE (left right)	2 1 3	2 1 3
A2 A3 A1 A4 D1 PRE (right right)	3 2 4	3 2 4
A3 A2 A4 A1 D4 PRE (left left)	2 1 3	2 1 3
A2 A4 A1 A3 D1 PRE (right left)	3 2 4	3 2 4
A3 A1 A4 A2 D4 PRE (left right)	2 1 3	2 1 3